# SC4020 Data Analytics & Mining
# Project Report 2

Group 44


| Name | Matriculation No. | Signature |
|---|---|---|
| Choo Yi Ken | U2240710B | |
| Lai Jun You | U2340700L | |
| Liaw Rui Xian | U2320440D | |
| Park Yumin | U2123691G | |



**Abstract:**
*This project applies pattern mining techniques to biomedical datasets for disease analysis and diagnosis. In Task 1, we implemented the Apriori algorithm with adaptive minimum support thresholds to identify symptom co-occurrence patterns across multiple diseases. In Task 2, we applied Generalized Sequential Pattern (GSP) mining to the Breast Cancer Wisconsin dataset, transforming continuous features into ranked categorical sequences to distinguish malignant from benign cases. Sensitivity analysis across three discretization strategies (quantile, uniform, k-means) assessed pattern stability. In Task 3, we developed a RAG-based medical chatbot that integrates discovered symptom patterns with large language model capabilities to provide interpretable disease predictions from user-reported symptoms.*

# 1. Analysis of Symptom Co-occurrence Patterns

## 1.1. Methodology and Implementation Choices

### 1.1.1. Data Standardization

To ensure the accuracy of the analysis, the raw symptom data was standardized. The data was identified to contain inconsistencies, including typos and formatting variations. `synonym_map` dictionary was created to map all known variations to a single, consistent term. This map was applied using a custom function `clean_symptom` , which also stripped whitespace and handled null values.

### 1.1.2. Binary Transaction Model

We chose to focus only on the presence of symptoms as a binary problem. This model was implemented in the function `prepare_disease_basket` which converts each patient's list of symptoms into a one-hot encoded vector.

### 1.1.3. Per-Disease Analysis

In order to analyse disease-specific symptom co-occurrence patterns, we first group all patient symptom lists by their diagnosed disease . The main analysis loop is implemented to iterate through each disease independently to run Apriori algorithm on each disease's subset of patients. Since the dataset provides an equal number of 120 transactions per disease, it already ensures balanced representation for each class. Therefore, the mirrored symptom dataset was not required, as this balance eliminates potential bias caused by unequal sample sizes across diseases.

### 1.1.4. Algorithmic Logic

The manual implementation of the Apriori algorithm can be found as `apriori_manual` function. The algorithm is designed to iteratively find frequent itemsets, starting with single symptoms and calculating their support. It then prunes any symptom that is below a `min_sup` threshold and joins the remaining frequent symptoms to create candidate pairs. This process is repeated until no new frequent itemsets can be found. We implement a key custom heuristic through the function `get_adaptive_minsupport`. Instead of one fixed or pre-determined `min_sup` for all diseases, the threshold was dynamically adjusted based on the number of unique symptoms for that disease. Such implementation allows us to find meaningful results in both complex and simple diseases without sacrificing computational efficiency.

## 1.2. Key Co-occurrence Patterns

The analysis successfully identified high-support symptom combinations for each disease. In the final step, the results were filtered to retain only co-occurrence patterns, allowing the identification of the most frequent symptom combinations. One of the highest-support patterns discovered for each disease is summarized in **Table 1**, while the complete set of patterns derived from the analysis is demonstrated in the accompanying explanation video.

| Disease | Frequent Itemset (Highest Support) | Support |
|---|---|---|
| Paroxysmal Positional Vertigo | {vomiting, unsteadiness} | 0.90 |
| AIDS | {extra_marital_contacts, high_fever} | 0.85 |
| Acne | {skin_rash, scarring} | 0.85 |
| Alcoholic hepatitis | {yellowish_skin, fluid_overload} | 0.90 |
| Allergy | {continuous_sneezing, shivering} | 0.80 |
| Arthritis | {stiff_neck, painful_walking} | 0.90 |
| Bronchial Asthma | {family_history, breathlessness} | 0.90 |
| Cervical spondylosis | {loss_of_balance, neck_pain} | 0.90 |
| Chicken pox | {malaise, red_spots_over_body} | 1.00 |
| Chronic cholestasis | {itching, abdominal_pain} | 0.90 |
| Common Cold | {sinus_pressure, redness_of_eyes, chest_pain} | 1.00 |
| Dengue | {nausea, loss_of_appetite} | 1.00 |
| Diabetes | {polyuria, increased_appetite} | 1.00 |
| Dimorphic hemorrhoids (piles) | {bloody_stool, pain_in_anal_region} | 0.90 |
| Drug Reaction | {itching, spotting_urination} | 0.85 |
| Fungal infection | {itching, nodal_skin_eruptions} | 0.80 |
| GERD | {cough, stomach_pain} | 0.90 |
| Gastroenteritis | {diarrhoea, vomiting} | 0.85 |
| Heart attack | {vomiting, chest_pain} | 0.85 |
| Hepatitis A | {yellowing_of_eyes, mild_fever} | 1.00 |
| Hepatitis B | {receiving_blood_transfusion, yellowing_of_eyes} | 1.00 |
| Hepatitis C | {fatigue, family_history} | 0.90 |
| Hepatitis D | {nausea, joint_pain} | 0.90 |
| Hepatitis E | {coma, loss_of_appetite} | 1.00 |
| Hypertension | {loss_of_balance, lack_of_concentration} | 0.90 |
| Hyperthyroidism | {abnormal_menstruation, muscle_weakness, irritability} | 1.00 |
| Hypoglycemia | {slurred_speech, irritability} | 1.00 |
| Hypothyroidism | {swollen_extremities, enlarged_thyroid} | 1.00 |
| Impetigo | {red_sore_around_nose, skin_rash} | 0.90 |
| Jaundice | {itching, abdominal_pain} | 0.90 |
| Malaria | {muscle_pain, high_fever} | 0.95 |
| Migraine | {excessive_hunger, irritability} | 0.90 |
| Osteoarthritis | {joint_pain, painful_walking} | 0.90 |
| Paralysis (brain hemorrhage) | {vomiting, altered_sensorium} | 0.85 |
| Peptic ulcer disease | {passage_of_gases, internal_itching} | 0.90 |
| Pneumonia | {rusty_sputum, fast_heart_rate} | 1.00 |
| Psoriasis | {skin_rash, inflammatory_nails} | 0.90 |
| Tuberculosis | {malaise, yellowing_of_eyes, loss_of_appetite} | 1.00 |
| Typhoid | {high_fever, chills} | 1.00 |
| Urinary tract infection | {bladder_discomfort, continuous_feel_of_urine} | 0.90 |
| Varicose veins | {fatigue, cramps} | 0.90 |

***Table 1: Results for Task 1***

# 2. Mining Cancer Feature Patterns

## 2.1. Data Preprocessing and Feature Transformation

The Breast Cancer Wisconsin dataset was loaded and preprocessed by removing non-feature columns ('`id`' and '`Unnamed: 32`'). All 30 continuous features were standardized using `StandardScaler` for z-score normalization to enable fair comparison across different scales, producing `df_standardized` for further analysis.

For sequential pattern mining, continuous features were discretized into 3 categorical levels ('`low`', '`mid`', '`high`') and stored in a dictionary, `discretized_datasets`. Three discretization strategies were employed, namely:

- **Quantile**: equal sample sizes per bin
- **Uniform**: equal bin widths
- **KMeans**: cluster-based binning

## 2.2. Patient Sequence Generation

A key step of the analysis was to convert each patient's static set of 30 features into an ordered sequence representing feature importance.

For each patient, the sequence construction proceeded as follows:

1. **Feature Ranking:** The absolute Z-scores of all features (from `df_standardized`) were computed, and percentile ranks were assigned using `scipy.stats.percentileofscore` to measure the relative deviation from the population mean, irrespective of direction.
2. **Feature Selection:** The five features with the highest percentile ranks were selected to form a sequence.
3. **Tokenisation:** Each selected feature was paired with its corresponding categorical level (e.g., `low`, `mid`, `high`), resulting in tokens such as '`radius_mean=high`'.
4. **Itemset Grouping:** Features with closely similar percentile ranks (within a small threshold) were grouped into a single itemset. The itemset grouping rule is that features are grouped only if their percentile ranks are very close (within a small threshold). However, given the distribution of the data, this resulted in sequences of individual itemsets, one per selected feature.

Before mining, the sequences were formatted and split by diagnosis. Each nested list of itemsets was converted into a flat list of strings. Using the sequences from the '`quantile`' discretization strategy as the primary dataset, two separate databases were created: Malignant (212 sequences) and Benign (357 sequences). This separation allows for the discovery of patterns that are uniquely characteristic of each diagnosis.

## 2.3. Sequential Pattern Mining

**Generalised Sequential Pattern (GSP) algorithm** is used for sequential pattern mining. GSP is an extension of the Apriori algorithm designed to handle sequences. It works by first identifying frequent individual items, then combining them to find longer

frequent sequences, pruning any sequences that fall below a minimum support threshold.

A `GSPMiner` class was implemented to perform the Generalized Sequential Pattern mining. The algorithm follows the Apriori principle:

- It first scans the database to find all frequent 1-item sequences (L1) that meet the minimum support threshold.
- It then iteratively generates candidate (k)-sequences from the frequent (k-1)-sequences found in the previous pass.
- Candidates are pruned if all their (k-1)-subsequences are not frequent.
- The database is scanned again to count the support of the remaining candidates, identifying the frequent k-sequences ($L_k$).

This process was repeated up to a maximum pattern length of 3. The mining was performed independently on the Malignant and Benign databases using a minimum support ratio of 10%.

### 2.3.1 Computational Complexity and Performance

Several design decisions were made to manage computational efficiency:

1. **Top-5 Feature Selection:** Each patient sequence was limited to the 5 most extreme features by percentile rank. This reduces the per-patient sequence length from a maximum of 30 to 5 items, which decreases the pattern mining complexity as shorter sequences generate fewer candidate patterns.
2. **Restriction on maximum pattern length = 3**: Restricting pattern length prevents exponential candidate growth. With 22 frequent 1-items discovered, the potential candidate space will explode exponentially at level 2. However, Apriori-based pruning reduced the amount of actual candidates being tested.

Mining Results: The algorithm converged efficiently:

| Level (Length) | Malignant Patterns | Benign Patterns |
|---|---|---|
| 1 | 22 | 22 |
| 2 | 3 | 1 |
| 3 | 0 | 0 |

***Table 2: Mining Results for Task 2***

Therefore, the parameters (`k=5`, `support=0.1`, `max_length=3`) balance pattern completeness with efficiency and interpretability. Lower support would yield more patterns but increase noise; higher k would provide more features but reduce interpretability.

## 2.4. Discovered Patterns from Mining

Sequential pattern mining revealed contrasting characteristics between malignant and benign cases. **Table 3** presents the top 5 most frequent patterns discovered for each diagnosis type (bracketed value represents the support ratio):

| Rank | Malignant Patterns | Benign Patterns |
|---|---|---|
| 1 | radius_mean=high (28.3%) | texture_worst=low (25.5%) |
| 2 | radius_worst=high (23.1%) | texture_mean=low (23.5%) |
| 3 | area_mean=high (22.2%) | smoothness_worst=low (23.5%) |
| 4 | compactness_worst=high (21.2%) | smoothness_mean=low (22.7%) |
| 5 | perimeter_mean=high (20.3%) | symmetry_mean=low (21.8%) |

***Table 3: Patterns discovered for Task 2***

Malignant cases consistently exhibited high values for size-related features (radius, area, perimeter), while benign showed low values for texture and smoothness features.

## 2.5. Sensitivity Analysis Across Discretisation Strategy

To assess the robustness of the discovered patterns, the entire mining process was repeated using the '`uniform`' and '`kmeans`' discretization strategies from `discretized_datasets`. Top 5 patterns across all three binning methods are compared for malignant patterns and benign patterns.

**Malignant Patterns:** Quantile and KMeans consistently identified 'high' size-related features (radius_mean=high, radius_worst=high) as dominant patterns. Uniform binning produced 'mid' level patterns, suggesting it splits clinically significant 'high' values across bins, diluting support.

**Benign Patterns:** Highly stable across all strategies. The top four patterns (texture_worst=low, texture_mean=low, smoothness_worst=low, smoothness_mean=low) remained identical with consistent support, indicating these are robust characteristics independent of the binning method.

```
COMPARISON: MALIGNANT Patterns Across Strategies

QUANTILE:
------------------------------------------------------------
  1. radius_mean=high                              |  60/212 (28.3%)
  2. radius_worst=high                             |  49/212 (23.1%)
  3. area_mean=high                                |  47/212 (22.2%)
  4. compactness_worst=high                        |  45/212 (21.2%)
  5. perimeter_mean=high                           |  43/212 (20.3%)

UNIFORM:
------------------------------------------------------------
  1. radius_mean=mid                               |  50/212 (23.6%)
  2. area_mean=mid                                 |  40/212 (18.9%)
  3. radius_worst=mid                              |  38/212 (17.9%)
  4. compactness_worst=mid                         |  35/212 (16.5%)
  5. perimeter_mean=mid                            |  35/212 (16.5%)

KMEANS:
------------------------------------------------------------
  1. radius_mean=high                              |  58/212 (27.4%)
  2. radius_worst=high                             |  45/212 (21.2%)
  3. perimeter_mean=high                           |  40/212 (18.9%)
  4. texture_worst=high                            |  39/212 (18.4%)
  5. area_mean=mid                                 |  35/212 (16.5%)
```

```
COMPARISON: BENIGN Patterns Across Strategies

QUANTILE:
------------------------------------------------------------
  1. texture_worst=low                             |  91/357 (25.5%)
  2. texture_mean=low                              |  84/357 (23.5%)
  3. smoothness_worst=low                          |  84/357 (23.5%)
  4. smoothness_mean=low                           |  81/357 (22.7%)
  5. symmetry_mean=low                             |  78/357 (21.8%)

UNIFORM:
------------------------------------------------------------
  1. texture_worst=low                             |  91/357 (25.5%)
  2. texture_mean=low                              |  84/357 (23.5%)
  3. smoothness_worst=low                          |  84/357 (23.5%)
  4. smoothness_mean=low                           |  81/357 (22.7%)
  5. texture_se=low                                |  79/357 (22.1%)

KMEANS:
------------------------------------------------------------
  1. texture_worst=low                             |  91/357 (25.5%)
  2. texture_mean=low                              |  84/357 (23.5%)
  3. smoothness_worst=low                          |  84/357 (23.5%)
  4. smoothness_mean=low                           |  81/357 (22.7%)
  5. symmetry_mean=low                             |  78/357 (21.8%)
```

***Screenshot 1: Results for sensitivity analysis (Left: Malignant, Right: Benign)***

# 3. Open Advanced Tasks

For the open advanced task, we developed a symptom-based diagnosis chatbot using a Retrieval-Augmented Generation (RAG) system built upon a structured medical knowledge base with the datasets used in task 1. The goal is to provide accurate, context-aware, and empathetic responses to user queries, ensuring all answers are grounded in verified medical data. The implementation consists of five major stages:

1. **Loading Data:** Importing and combining four medical datasets — *disease_desc.csv*, *symptoms_desc.csv*, *precaution.csv*, and *itemsets.csv*.
2. **Pre-processing and Integration:** Cleaning and merging datasets to form a unified medical knowledge base.
3. **Creating LangChain Documents:** Formatting the textual data from the knowledge base into LangChain Document objects, making it ready for processing.
4. **Embedding and Indexing:** Encoding the Documents into vector representations (embeddings) and loading them into a FAISS index to enable efficient semantic search.
5. **Retrieval-Augmented Question Answering:** Executing the RAG chain, which takes a user's question, uses the FAISS index to retrieve relevant context, and then passes that context to the LLM to generate an accurate answer.

## 3.1 Data Loading and Preparation

The pipeline begins by importing essential libraries, including LangChain components, FAISS, Hugging Face embeddings, and Google's Gemini API. The four CSV files are then loaded using pandas and stored as DataFrames. The file `disease_desc.csv` provides detailed textual descriptions of various diseases. The file `symptoms_desc.csv` contains descriptions for each symptom, including both generic and medical definitions. The file `precaution.csv` lists suggested actions or precautions for each disease, while `itemsets.csv` contains the top 20 frequent symptom sets associated with diseases, mined using the Apriori algorithm from Task 1.

## 3.2 Data Pre-processing and Merging

The pipeline performs a crucial data transformation step to convert the `itemsets.csv` file from a "wide" format to a "long" format. This transformation is achieved using the `pd.melt` function, which links each disease to multiple symptom item sets in a more structured and uniform manner. To clean and interpret the symptom lists, the function `parse_symptom_string` is defined. It removes brackets and whitespace, filters out numerical values that may appear in the symptom sets, and returns a clean list of symptom names. Once this process is complete, the system groups all itemsets by disease and unifies them into a single list of unique symptoms for each disease. The result is a dictionary mapping each disease to all its associated symptoms. For

example: `'(vertigo) Paroymsal Positional Vertigo': {'nausea', 'loss_of_balance', 'spinning_movements', 'unsteadiness', 'headache', 'vomiting'}`

## 3.3 Document Creation for LangChain

Each disease is converted into a LangChain Document object, which includes the disease description, a list of common symptoms (with their individual explanations), and a list of recommended precautions. Each document also includes metadata, (disease name and a data source tag). The document for "AIDS" look like this:

```
--- Example LangChain Document ---

Disease: AIDS
Description: Acquired immunodeficiency syndrome (AIDS) is a chronic, potentially life-threatening condition caused by the human immunodeficiency virus (HIV). By damaging your

Common Symptoms (derived from frequent itemsets):
  - Patches_in_throat: Spots, streaks or areas of discoloration (often white or yellow) on the throat or tonsils, commonly caused by viral, bacterial or fungal infections.
  - Muscle_wasting: the process of losing muscle mass, which leads to decreased muscle strength and function
  - High_fever: A body temperature significantly above normal (e.g., above ~38?°C for adults) resulting from the body@s immune response to infection or illness.
  - Extra_marital_contacts: intimate sexual contacts outside of a committed (married) relationship

Precautions:
  - avoid open cuts
  - wear ppe if possible
  - consult doctor
  - follow up

Metadata: {'disease': 'AIDS', 'source': 'medical_db_v2'}
```

***Screenshot 2: Example LangChain Document for disease 'AIDS'***

## 3.4 Creating Vector Store

A local sentence transformer model `all-MiniLM-L6-v2` is used to generate vector representations of documents. These document vectors are then indexed into a FAISS vector store to create an efficient system for high-speed similarity searches. To query this store, a retriever is defined to fetch the top three most relevant documents for any given query.

**Test Case:**

- **Query:** "I have a high temperature and my whole body hurts"
- **Result:** The retriever searches the FAISS store and returns the three diseases whose descriptions most closely match the query's vector, such as: Allergy, Typhoid, and Common Cold.

## 3.5 RAG Chain Design

The Gemini 2.5 Flash model is initialized with a low temperature (0.3) to ensure consistent and factual responses. A structured prompt template defines the chatbot's behavior, instructing it to respond empathetically while using only the retrieved contextual information. The final RAG chain combines this model with the retriever to generate accurate, context-grounded medical answers.

## 3.6 Chatbot Interface and Testing

A helper function `ask_chatbot` is defined to send user queries to the RAG chain and display the chatbot's responses along with the matched diseases retrieved. To validate its performance, we tested the system with a comprehensive test case covering diverse user queries, including: direct symptom descriptions, ambiguous symptom descriptions, questions about a particular disease, completely unrelated (out-of-scope) questions. This helped us confirm that the chatbot could identify the right disease, provide helpful precautions based on the context, and gracefully handle queries it could not answer. Table below shows examples of user queries and the corresponding chatbot responses, illustrating both successful matches and no-match scenarios.

| **User Query** | **Chatbot Response** |
|---|---|
| I have a high temperature, cough, and my whole body aches. What should I do? | I'm sorry you're feeling unwell. From what you've described – a high temperature, cough, and whole body aches – it sounds quite similar to symptoms associated with a **Common Cold**.<br><br>The common cold is a viral infection that affects your nose and throat, which is part of your upper respiratory tract. While it can certainly make you feel miserable, it's generally considered harmless. Many different types of viruses can be responsible for causing a common cold.<br><br>To help take care of yourself, the information suggests:<br>* Drinking vitamin C rich drinks<br>* Taking vapour<br>* Avoiding cold food<br>* Keeping your fever in check<br><br>Remember, this is general information from the provided source — it's always best to check with a healthcare professional for proper diagnosis and care.<br><br>--- (Debug) Retrieved Documents: ---<br>- Matched Disease: Allergy<br>- Matched Disease: Common Cold<br>- Matched Disease: Typhoid |
| My Wi-Fi isn't working properly. | It sounds like you're having trouble with your Wi-Fi, and I understand how frustrating that can be! However, the information I have is about medical conditions like Diabetes, the Common Cold, and Paralysis.<br><br>Since your query is about a technical issue with your Wi-Fi and not a medical symptom, I don't have any relevant information in my medical database to help with that.<br><br>Remember, I can only provide information based on the medical context I've been given.<br><br>--- (Debug) Retrieved Documents: ---<br>- Matched Disease: Diabetes<br>- Matched Disease: Common Cold<br>- Matched Disease: Paralysis (brain hemorrhage) |

***Table 3: Results for Task 3***